

Tourism Government — Detailed Testing Guide

Complete Line-by-Line Code Explanations of JUnit 5 & Mockito Unit Tests

■■ SAFEGUARD ASSURANCE:

Please rest assured that **NO business logic, DTOs, Controllers, Services, or database tables were removed or deleted** from your code! All your production code remains 100% untouched and functional. The only files created were lightweight unit test classes inside your `src/test/java` directories to help you clear your technical interviews.

1. Core Concepts: What is Unit Testing & Mocking?

Unit testing means testing the smallest piece of code (a single Java class or method) in absolute isolation. In a microservices architecture, our services depend on repositories and other services. Instead of running a real database or calling a real network REST client, we use **Mockito** to mock these dependencies.

- **Mock (@Mock):** A fake object that replaces a real class. It behaves like the real class but does nothing unless we tell it to.
- **InjectMocks (@InjectMocks):** Creates the real service implementation we want to test and automatically plugs in all our fake mocked objects into it.
- **Stubbing (when/thenReturn):** Programming a mock object to return a pre-determined mock response. It's like pre-recording a response.

2. Line-by-Line Walkthrough: NotificationServiceTest.java

• setUp() Method

This method runs before every single test case. We initialize a fake `NotificationRequestDTO` (input data), a fake `UserDTO` (user details), and a fake `Notification` (expected output).

```
requestDTO = NotificationRequestDTO.builder().userId(1L).subject("Test").build();
```

• testCreateNotification_Success()

1. **Stubbing:** we tell Mockito `when(userClient.getUserById(1L)).thenReturn(mockUser)`. So when the service requests user details, it gets our mock user.
2. **Stubbing repository:** we tell Mockito `when(notificationRepository.saveAndFlush(any())).thenReturn(mockNotification)` so it returns our pre-cooked notification object.
3. **Service execution:** We call the real service method `notificationService.create(requestDTO)`.
4. **Verification:** We check that the result is not null, has our expected ID 10L, and verify that our mocked repositories and email services were actually invoked.

```
when(userClient.getUserById(1L)).thenReturn(mockUser);  
when(notificationRepository.saveAndFlush(any())).thenReturn(mockNotification);  
NotificationResponseDTO result = notificationService.create(requestDTO);  
assertEquals(10L, result.getNotificationId());
```

- **testCreateNotification_UserNotFound()**

This tests our fault-tolerant fallback code. We stub `userClient` to throw an exception to simulate a service timeout. We assert that the service does NOT crash, but instead proceeds gracefully using default values and completes successfully.

```
when(userClient.getUserById(1L)).thenThrow(new RuntimeException("Offline"));
NotificationResponseDTO result = notificationService.create(requestDTO);
assertNotNull(result);
```

3. Line-by-Line Walkthrough: ReportServiceTest.java

The `ReportServiceImpl` contains complex access rules and external calls across five Feign clients. We test these rules in isolation.

- **testGenerateReport_Success()**

We stub `UserClient` to return an `ADMIN` role. We stub the `SiteClient` to return an empty list of sites (representing the data scope). We then assert that the generated report is saved and has the correct creator name.

```
when(userClient.getUserById(1L)).thenReturn(mockRequester);
when(siteClient.getAllSites()).thenReturn(Collections.emptyList());
ReportSummaryDTO result = reportService.generateReport(requestDTO);
assertEquals("Omkar Admin", result.getGeneratedByName());
```

- **testGenerateReport_DeniedForTourist()**

This tests our security access controls. We change the mock requester's role to `TOURIST`. Since tourists are not allowed to generate reports, we assert that the service correctly throws a `ResponseStatusException` (HTTP 403 Forbidden).

```
mockRequester.setRole(Role.TOURIST);
when(userClient.getUserById(1L)).thenReturn(mockRequester);
assertThrows(ResponseStatusException.class, () -> {
    reportService.generateReport(requestDTO);
});
```

4. How to confidently explain this testing to an interviewer

■■ Your Verbal Pitch:

"We implement unit tests using JUnit 5 and Mockito to verify our core business domains. In our Notification and Reporting modules, we mock out database repositories and external Feign client interfaces. This allows us to test the service algorithms, access controls, and boundary validations in absolute isolation without requiring a database connection or network calls, executing our entire test suite in milliseconds and ensuring high code quality and stability."